

[NAME OF THE COLLEGE / INSTITUTE]

Department of Computer Science / Information Technology

PROJECT REPORT

AI Smart Inventory Management System

A full-stack web application for inventory management, GST billing, customer credit tracking and AI-driven business insights, developed using the MERN technology stack.

Submitted by: _____ Roll No.: _____

Under the guidance of: _____

Academic Year 2025 – 2026

Abstract

The AI Smart Inventory Management System is a full-stack web application designed to digitise the day-to-day operations of small and medium retail businesses. A large proportion of small retailers continue to manage stock in paper registers, prepare bills manually, and record customer credit in physical ledgers. These practices result in calculation errors, lost records, poor tax compliance and an absence of meaningful business insight.

The proposed system addresses these problems through a single integrated platform built on the MERN stack (MongoDB, Express.js, React and Node.js). It provides real-time inventory tracking, rapid point-of-sale billing with automatic GST computation, server-generated PDF invoices with UPI payment QR codes, a digital customer credit ledger, and supplier management. The system is further enhanced with artificial intelligence: the Google Gemini API is used for demand forecasting and reorder recommendations, while Tesseract.js provides optical character recognition for importing products directly from photographed supplier bills. Scheduled background jobs generate periodic reports and raise smart stock alerts automatically.

The application follows a layered architecture with validated REST APIs, JWT-based authentication, and precise decimal arithmetic for all monetary calculations. The result is a reliable, extensible and practical system that demonstrates the complete software development life cycle, from requirement analysis and database design to implementation and testing.

1. Introduction

Inventory management is the backbone of every retail business. While large enterprises employ sophisticated ERP systems, the vast majority of small retailers — grocery stores, traders and wholesalers — continue to depend on manual methods. Stock registers are updated by hand, bills are calculated mentally or on a calculator, and customer credit is maintained in a paper ledger commonly known as a *khata*.

This project, the **AI Smart Inventory Management System**, was developed to bring the benefits of modern software and artificial intelligence to such businesses in a form that is affordable, simple and aligned with their actual daily workflow. The system unifies inventory, billing, customer credit, supplier records, analytics and reporting into one web application, and applies AI where it produces direct business value: predicting demand, recommending reorders and automating routine analysis.

1.1 Problem Statement

- **Manual stock-keeping:** stock levels recorded on paper are frequently inaccurate, and shortages are discovered only when a customer asks for an unavailable item.
- **Error-prone billing:** manual GST computation leads to incorrect tax amounts, non-compliant invoices and revenue leakage.
- **Unreliable credit records:** paper credit ledgers are easily lost or disputed, and outstanding amounts are difficult to total or follow up.
- **No business insight:** the owner has no data on fast-moving products, seasonal demand, stock shrinkage or profitability.
- **Unsuitable existing software:** commercial solutions are often expensive, overly complex, or not designed around the workflow of small retail businesses.

1.2 Objectives

1. To digitise inventory with real-time stock levels, units of measure, low-stock detection and shrinkage (stock-adjustment) tracking.
2. To provide a fast point-of-sale billing module with automatic GST calculation, PDF invoice generation and UPI QR codes for digital payment.
3. To replace the paper credit ledger with a digital *khata* that maintains an accurate running balance for every customer.
4. To integrate artificial intelligence for demand forecasting, reorder recommendation and an in-application help assistant.
5. To automate routine work — periodic reports and stock alerts — using scheduled background jobs.
6. To deliver analytics dashboards and exportable reports, including a Tally-compatible export for accountants.

7. To ensure correctness and security through input validation, authenticated APIs and precise decimal arithmetic for money.

2. System Analysis

2.1 Existing System

In the existing manual system, every business activity depends on human memory and handwriting. Purchases are noted in a register, sale bills are written by hand or generated from a basic calculator, and credit transactions are entered into a physical ledger. There is no linkage between these records: a sale does not automatically reduce stock, a credit bill does not automatically update the customer's balance, and no consolidated reporting exists. Errors accumulate silently and the true state of the business is never known with certainty.

2.2 Proposed System

The proposed system replaces these disconnected manual records with a single integrated application in which every operation updates all related data automatically. When a sale is completed, stock is reduced, the invoice is generated, the payment is recorded, and — if the sale is on credit — the customer's ledger is updated, all in one atomic workflow. The system additionally analyses accumulated data to forecast demand and alert the owner before stock runs out.

2.3 Advantages of the Proposed System

- Single source of truth for stock, sales, customers, suppliers and credit.
- Accurate, GST-compliant invoices produced in seconds.
- Elimination of disputes over customer credit through a permanent digital ledger.
- Data-driven purchasing decisions through AI demand forecasting and reorder reports.
- Reduced data-entry effort through OCR-based import of supplier bills.
- Automatic alerts and reports without any manual effort.

3. System Architecture

The application follows a three-tier architecture. The presentation tier is a React single-page application; the application tier is a Node.js/Express REST API organised into routes, controllers and services; and the data tier is a MongoDB database accessed through the Mongoose object-document mapper.

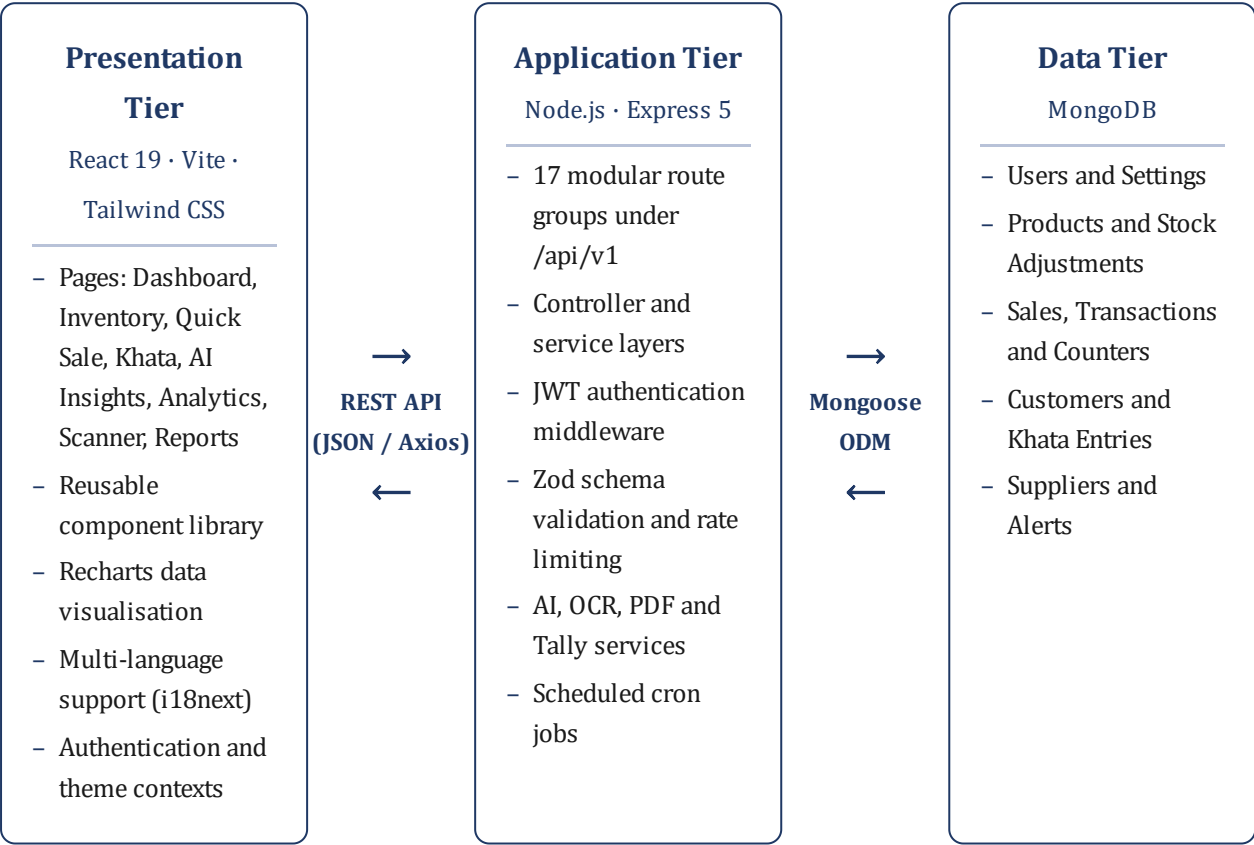


Figure 3.1 — Three-tier architecture of the AI Smart Inventory Management System

On the server, every request passes through a defined pipeline: authentication middleware verifies the JSON Web Token, validation middleware checks the request body against a Zod schema, the controller orchestrates the operation, and the service layer contains the business logic and database access. A central error-handling middleware converts all failures into consistent JSON error responses.

4. Technology Stack

Layer	Technology	Purpose
Frontend	React 19, Vite, Tailwind CSS	Responsive single-page user interface
Visualisation	Recharts	Sales trends, demand charts and analytics
Internationalisation	i18next, react-i18next	Multi-language user interface
Backend	Node.js, Express 5	RESTful API with layered architecture
Database	MongoDB, Mongoose 9	Document data store; Decimal128 for monetary values
Artificial Intelligence	Google Gemini API	Demand forecasting, insights and help assistant
Optical Character Recognition	Tesseract.js	Extraction of product data from supplier bill images
Document Generation	PDFKit, qrcode	GST invoices with embedded UPI payment QR codes
Security	JSON Web Tokens, bcrypt, Zod, express-rate-limit	Authentication, password hashing, input validation and abuse protection
Task Scheduling	node-cron	Automated smart alerts and report generation
File Handling	Multer	Upload of bill images for OCR processing
Testing	Node.js test runner, Axios	API smoke tests and end-to-end tests

5. Module Description

5.1 Authentication and Onboarding

Users register and sign in with credentials that are hashed using bcrypt before storage. Successful authentication issues a JSON Web Token, which is required by every protected API route. A guided seven-step onboarding wizard collects the business profile, GST details, payment preferences, initial products, opening stock and supplier information, and concludes with the user's first invoice, ensuring that a new business is fully operational within minutes.

5.2 Inventory and Product Management

The inventory module maintains the product catalogue with categories, pricing, applicable GST rate and unit of measure (pieces, kilograms, grams or litres). Stock levels are updated automatically by every sale and purchase. Configurable reorder thresholds enable low-stock detection, and a dedicated stock-adjustment sub-module records shrinkage — damage, theft, expiry or counting errors — together with the reason, so losses can be analysed rather than silently absorbed. Monetary amounts and weights are stored using MongoDB's Decimal128 type, with utility modules guaranteeing that floating-point rounding errors cannot occur.

5.3 Quick Sale and GST Billing

The billing module is designed around the speed of a real shop counter. The operator searches for a product, enters a weight or an amount, and the bill is constructed automatically. An amount-first mode back-calculates the quantity when a customer asks for a fixed value of goods, and tare-weight support handles weighing with containers. GST is computed per line item according to each product's tax rate. Payment may be recorded as cash, online (UPI), or credit; credit sales flow directly into the customer's digital ledger. On completion the server generates a PDF invoice using PDFKit, embedding a UPI QR code that the customer can scan to pay instantly.

5.4 Customer Credit Ledger (Digital Khata)

The khata module is a faithful digital replacement for the traditional paper credit ledger. Every credit sale automatically creates a ledger entry against the customer, and

repayments — full or partial — are recorded in the same timeline. A service layer maintains a consistent running balance per customer, and the owner can view the total outstanding credit across the entire customer base at any time. Because the history is permanent and timestamped, disputes over amounts owed are eliminated.

5.5 Supplier Management

Supplier records, purchase transactions and supplier-related shrinkage (sample packs) are managed in a dedicated module, giving the owner a complete picture of the procurement side of the business.

5.6 AI Insights and Demand Forecasting

The intelligence layer of the system is built on the Google Gemini API. The AI service assembles the business's sales and stock history into a structured prompt and obtains demand forecasts and reorder recommendations, which are presented on the AI Insights page as charts and actionable suggestions. A reorder report answers the owner's most important question — what to restock, in what quantity, and when — before items run out. An integrated help assistant additionally answers usage questions inside the application, lowering the barrier for non-technical users.

5.7 OCR Bill Scanner

Entering dozens of items from a supplier bill by hand is slow and error-prone. The scanner module allows the owner to photograph and upload the bill; the image is processed on the server by Tesseract.js, and the OCR service parses product names, quantities and prices from the recognised text. The parsed items are presented to the user for review and can be imported into inventory with a single action.

5.8 Analytics, Reports and Alerts

A dashboard summarises key performance indicators — daily sales, outstanding credit, low-stock count and top-selling products. The analytics module charts sales trends, category performance and payment-mode distribution. Two scheduled jobs run automatically: a report generator that compiles periodic business reports, and a smart-alerts job that scans inventory and raises notifications for low stock and unusual activity. Transaction data can also be exported in a Tally-compatible format for the business's accountant.

6. Database Design

The database consists of twelve principal collections. The most significant collections and their roles are summarised below.

Collection	Description
User	Account credentials (hashed), business profile and role information.
Settings	Business configuration: GST details, invoice preferences, payment options.
Product	Catalogue item with category, price, GST rate, unit of measure, stock level and reorder threshold.
Sale	Completed bill with line items, quantities, GST breakdown, totals and payment mode.
Transaction	Stock movement records for purchases and sales.
StockAdjustment	Shrinkage entries with quantity, reason and date.
Customer	Customer directory with contact details and credit summary.
KhataEntry	Individual credit or repayment entry linked to a customer and, where applicable, a sale.
Supplier	Supplier records and procurement details.
Alert	System-generated notifications (low stock, anomalies).
Counter	Atomic sequence generator for invoice numbering.

All monetary fields use the Decimal128 data type rather than floating-point numbers, ensuring that invoice totals and GST amounts are exact. Invoice numbers are generated from an atomic counter collection so that numbering remains sequential and gap-free even under concurrent requests.

7. Security and Software Quality

- **Authentication:** stateless JWT sessions; passwords hashed with bcrypt; protected routes enforced by middleware on both the API and the user interface.
- **Input validation:** every endpoint validates its request body against a Zod schema through a central validation middleware, so malformed or malicious data never reaches the database layer.
- **Rate limiting:** express-rate-limit protects authentication and API endpoints against brute-force attacks and abuse.
- **Monetary precision:** all currency and weight arithmetic is performed with decimal libraries and stored as Decimal128, eliminating floating-point rounding errors in bills and tax calculations.
- **Error handling:** a central error middleware on the server produces consistent error responses, while React error boundaries and toast notifications provide graceful failure handling on the client.
- **Testing:** automated smoke tests and end-to-end tests, executed with the Node.js test runner, verify endpoint behaviour, response structure, status codes and the correctness of GST computations.

8. Future Scope

1. **Mobile application:** a React Native or progressive web application so that billing can be performed on any phone at the counter.
2. **Messaging integration:** automatic delivery of invoices and credit-payment reminders through WhatsApp or SMS.
3. **Barcode scanning:** camera-based barcode support for rapid billing of packaged goods.
4. **Multi-store and multi-user support:** workspaces with role-based permissions for staff members across multiple branches.
5. **Offline-first operation:** local storage with background synchronisation so that billing continues during internet outages.

6. **Advanced intelligence:** price optimisation, festival-season demand models and anomaly detection for identifying theft patterns.

9. Conclusion

The AI Smart Inventory Management System successfully digitises the complete daily workflow of a small retail business — inventory, billing, customer credit, supplier management and analysis — within a single coherent application. The integration of artificial intelligence is practical rather than decorative: demand forecasts, reorder recommendations and automated alerts directly answer the questions a business owner asks every day.

The project demonstrates the full breadth of modern full-stack engineering: requirement analysis, layered system architecture, document database design, secure and validated REST APIs, a contemporary React user interface, integration of AI and OCR services, scheduled automation, and automated testing. Deliberate engineering decisions — exact decimal arithmetic for money, schema-validated inputs, token-based security and atomic invoice numbering — make the system dependable enough for real commercial use, and its modular architecture provides a solid foundation for the enhancements outlined in the future scope.

References

1. React Documentation — <https://react.dev>
2. Express.js Documentation — <https://expressjs.com>
3. MongoDB and Mongoose Documentation — <https://www.mongodb.com> / <https://mongoosejs.com>
4. Google Gemini API Documentation — <https://ai.google.dev>
5. Tesseract.js Documentation — <https://tesseract.projectnaptha.com>
6. PDFKit Documentation — <https://pdfkit.org>
7. Tailwind CSS Documentation — <https://tailwindcss.com>